

Graph_Attention_and_DRL_for_Task_Offloading_in _Industrial_Image_Recognition_over_Dynamic_Digi tal_Twin_Edge_Networks (1).pdf

by Dung Do Ngoc

Submission date: 16-Jun-2026 12:59PM (UTC+0800)

Submission ID: 2972855924

File name: Graph_Attention_and_DRL_for_Task_Offloading_in_Industrial_Image_Recognition_over_Dynamic_Digital_Twin_Edge_Networks_1_.pdf (3.27M)

Word count: 7317

Character count: 41646

Dual Graph Attention and Action Masking for Industrial Image Recognition Task Offloading in Digital Twin Edge Networks

1st Dung Do

Department of Computer Science and Engineering

Yuan Ze University

Taoyuan, Taiwan

1146085@mail.yzu.edu.tw

Abstract—Task offloading in Digital Twin Edge Networks (DITENs) has become a critical enabler for supporting delay-sensitive and computation-intensive industrial applications such as image recognition. Existing approaches, including GCN-enabled multi-agent deep reinforcement learning frameworks, have shown promising performance in modeling task dependencies and optimizing delay-energy trade-offs. However, these methods still have limitations in distinguishing the varying importance of task dependencies and dynamic device-server connections in mobile industrial environments. To address these challenges, this paper proposes a graph-attention-enhanced task offloading framework that integrates Graph Attention Networks (GATs) with reinforcement learning. Unlike conventional approaches that mainly rely on raw or uniformly aggregated graph representations, the proposed method leverages GAT to dynamically capture complex relationships among subtasks, industrial devices, and edge servers. Specifically, GAT is used to extract attention-aware representations from both the task dependency graph and the network topology graph, while a coverage-aware action masking mechanism is adopted to remove infeasible offloading actions. Experimental results show that the proposed method outperforms the standard MAPPO model in terms of system reward, task processing delay, and energy consumption. Ablation results further verify the effectiveness of coverage-aware action masking and the additional benefit of GAT-enhanced representation.

Index Terms—Task Offloading, Digital Twin Edge Networks, Graph Attention Networks (GAT), Industrial Internet of Things (IIoT), Edge Computing, Deep Reinforcement Learning.

I. INTRODUCTION

With the rapid evolution of 6G communication and the Industrial Internet of Things (IIoT), industrial environments are increasingly relying on intelligent and delay-sensitive applications such as industrial image recognition, fault diagnosis, quality inspection, and predictive maintenance [1]. These applications generate massive amounts of computation-intensive data that often exceed the processing capability of resource-constrained industrial devices. To address this challenge, Digital Twin Edge Networks (DITENs) have emerged as a promising paradigm by integrating digital twin technology with edge computing, enabling real-time monitoring, low-latency processing, and intelligent task offloading in industrial systems. In particular, task offloading plays a critical role in reducing computation delay and energy consumption by

transferring complex tasks from industrial devices to nearby edge servers [2], [3].

Employing 0/1 offloading or partial offloading to balance task processing delay and energy consumption, the existing optimization methods are primarily designed for independent tasks [4]–[6]. However, with the evolution of industrial applications, tasks are increasingly composed of multiple subtasks exhibiting clear execution and data dependencies. Unlike independent-task offloading, dependent-task offloading must account for the sequential execution and data interactions among subtasks, significantly increasing the complexity of offloading. With this framework, the existing achievements have primarily focused on optimizing delay [7], energy consumption [8], or a combination of both [9]. Nevertheless, with the increasing number of subtasks, the complexity of dependent-task offloading is significantly intensified in dynamic environments, limiting the effectiveness and adaptability of existing solutions.

Recently, several studies have explored dependent-task offloading in DITEN-enabled industrial environments. Existing approaches commonly model industrial image recognition tasks as DAGs [1], where subtasks exhibit strong execution and data dependencies. To capture these dependencies, Graph Convolutional Networks (GCNs) combined with Deep Reinforcement Learning (DRL) have been introduced for optimizing offloading decisions. Furthermore, multi-agent reinforcement learning frameworks such as MADDPG have demonstrated promising performance in balancing task delay and energy consumption under dynamic industrial scenarios. However, these approaches still face limitations when representing the varying importance of different task dependencies and dynamic device-server connections. In practical industrial environments, not all subtasks, dependency edges, and edge-server connections contribute equally to the offloading decision. Therefore, a more expressive graph representation is required to support efficient decision-making.

The primary limitation of existing GCN-enabled DRL frameworks lies in their uniform neighborhood aggregation mechanism. Although GCNs are effective in extracting graph-structured information, they generally aggregate neighboring

features according to the graph structure without explicitly distinguishing the relative importance of different neighbors. For dependent-task offloading, this may limit the representation of critical subtasks and dependency paths in the DAG. Similarly, in the network topology, different edge servers may have different levels of importance due to coverage availability, connection status, waiting delay, and computing resources. If these relationships are not sufficiently differentiated, the DRL agent may receive less informative state representations, leading to inefficient exploration and suboptimal offloading decisions.

To overcome these challenges, this paper proposes a graph-attention-enhanced task offloading framework for industrial image recognition in DITENs. Unlike flat state approaches or uniformly aggregated graph representations, the proposed framework utilizes Graph Attention Networks (GATs) to dynamically learn the importance of relationships in both the task dependency graph and the network topology graph. Specifically, the task-level GAT is used to capture attention-aware dependencies among subtasks in the DAG, while the topology-level GAT is used to represent dynamic relationships between industrial devices and edge servers. By assigning different attention weights to different neighbors, the proposed method generates a richer and more adaptive state representation for reinforcement learning-based offloading.

In addition, a coverage-aware action masking mechanism is adopted to improve the feasibility and efficiency of offloading decisions. Since mobile industrial devices may move in and out of the coverage areas of edge servers, not all edge servers are valid execution locations at every time slot. Therefore, the action mask removes edge servers that are outside the current coverage range from the action space before the agent selects an action. This mechanism reduces invalid exploration and helps the agent focus on feasible offloading choices, thereby improving learning efficiency and decision quality in dynamic industrial edge environments.

Experimental results show that the proposed framework achieves improved performance in terms of system reward, task processing delay, and energy consumption compared with the standard MAPPO model. In addition, ablation studies verify the effectiveness of the task-level GAT, topology-level GAT, and coverage-aware action masking components. By leveraging attention-based graph representation and feasible action selection, the proposed model provides improved robustness and adaptability in industrial environments with mobile devices and dynamic network connections.

The main contributions are summarized as follows.

- 1) Following the GCN-enabled baseline formulation, a DITEN-based dependent-task offloading environment is adopted to evaluate delay-energy optimization for industrial image recognition with mobile devices and dynamic edge server connections.
- 2) By modeling the complex image recognition task as a DAG, a GAT-based task dependency representation and priority generation method is proposed to learn the relative importance of subtasks and dependency edges,

providing high-quality representations for offloading decisions.

- 3) A topology-level GAT is designed to capture the dynamic relationships between industrial devices and edge servers, enabling the offloading agent to better represent device-server connectivity, coverage availability, waiting delay, and edge network states.
- 4) A coverage-aware action masking mechanism is adopted to reduce infeasible offloading actions. By removing edge servers outside the coverage range from the action space, the proposed method improves learning efficiency and supports more reliable offloading decisions.

II. RELATED WORK

In dynamic industrial scenarios, task offloading has attracted significant attention due to the limited computing resources of industrial devices and the increasing complexity of IIoT applications. To improve service quality and reduce system cost, existing studies have investigated different offloading strategies for optimizing task processing delay, energy consumption, or their trade-off. For example, Lyapunov optimization, game-theoretic methods, heuristic algorithms, and reinforcement learning have been widely adopted to support computation offloading and resource allocation in mobile edge computing systems [10]. These methods provide effective solutions for conventional offloading problems. However, most of them mainly focus on independent tasks, where each task is treated as a complete and indivisible unit. Such an assumption may not be suitable for complex industrial applications, where a task is usually composed of multiple subtasks with explicit execution orders and data dependencies.

With the evolution of industrial applications, dependent-task offloading has become an important research direction. In industrial image recognition, a complete task can be divided into several subtasks, such as image extraction, denoising, standardization, feature extraction, and recognition. These subtasks are not independent, since the execution of one subtask may depend on the results generated by its predecessors. Therefore, dependent-task offloading must jointly consider subtask execution order, data transmission between subtasks, and the selection of execution locations [11]. To represent such dependency relationships, a task is commonly modeled as a directed acyclic graph (DAG), where nodes denote subtasks and edges denote dependency relations. Based on DAG modeling, several studies have optimized task processing delay [7], energy consumption [8], or delay-energy trade-offs [9]. Nevertheless, as the number of subtasks and dependency relations increases, the offloading decision space becomes significantly larger, making it difficult for conventional optimization and heuristic methods to obtain efficient decisions in dynamic industrial environments.

To address the complexity of dependent-task offloading, graph neural networks (GNNs) have been introduced to extract structural information from task DAGs. Since GNNs are effective in modeling graph-structured data, they can capture the dependency relationships among subtasks and

provide informative representations for offloading decision-making. Several studies have combined GNNs with deep reinforcement learning (DRL) to optimize task scheduling and offloading in edge computing systems. For instance, GNN-based methods have been used to learn subtask features from DAGs, while DRL algorithms such as DDPG and multi-agent DRL have been employed to determine offloading decisions. In particular, Graph Convolutional Networks (GCNs) have been applied to generate execution priorities of dependent subtasks, thereby improving the quality of state representations for reinforcement learning agents [1]. These studies demonstrate the potential of GNN-based methods for dependency-aware offloading. However, GCN-based representation mainly relies on neighborhood aggregation, which may not sufficiently distinguish the relative importance of different dependency edges and subtasks during decision-making.

Digital twin (DT) technology has also been explored to improve task offloading in industrial edge environments [12]. By constructing digital replicas of physical entities, such as industrial devices, tasks, and edge servers, DT can provide real-time monitoring and virtual-to-physical mapping for offloading decision support. In DT-assisted edge computing, the system can evaluate offloading strategies in the digital space before applying them to the physical environment, reducing the cost and risk of decision validation. Therefore, Digital Twin Edge Networks (DITENs), which integrate DT and edge computing, have become a promising paradigm for supporting low-latency and computation-intensive industrial applications. Existing DT-assisted offloading studies have considered resource allocation, task assignment, and intelligent decision-making in edge networks [13], [14]. However, the combination of DT, dependent-task offloading, and graph-based representation remains challenging due to the dynamic states of mobile industrial devices, changing network connections, and complex subtask dependencies.

Recently, Liao et al. [1] proposed a GCN-enabled task offloading framework for industrial image recognition in DITENs. Their work models image recognition tasks as DAGs, uses GCN to extract subtask dependency features and generate execution priorities, and designs an improved multi-agent DRL algorithm to optimize task processing delay and energy consumption in a multislot industrial scenario with mobile devices. This work provides a strong foundation for dependent-task offloading in DITEN-enabled IIoT environments. Nevertheless, its graph representation mainly depends on GCN-based neighborhood aggregation. Although GCN can capture graph structure, it does not explicitly assign different importance weights to different task dependencies or device-server connections. In practical industrial edge environments, not all subtasks, dependency edges, or edge-server connections contribute equally to the offloading decision. Moreover, due to device mobility, some edge servers may be outside the coverage range of an industrial device at a given time slot, making the corresponding offloading actions infeasible.

Motivated by these limitations, this paper extends the GCN-enabled dependent-task offloading framework by intro-

ducing graph attention into both task and network representations. Different from existing GCN-based methods that mainly rely on uniform neighborhood aggregation, the proposed framework uses Graph Attention Networks (GATs) to learn attention-aware representations from the task dependency graph and the dynamic network topology graph. The task-level GAT is used to capture the relative importance of subtasks and dependency edges in the DAG, while the topology-level GAT is used to represent dynamic relationships between industrial devices and edge servers. In addition, a coverage-aware action masking mechanism is introduced to remove infeasible offloading actions caused by device mobility and coverage limitations. By integrating GAT-based representation and feasible action selection, the proposed framework improves the learning efficiency and offloading decision quality in dynamic industrial edge environments.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we present the system model and problem formulation for dependent-task offloading in Digital Twin Edge Networks (DITENs). Following the GCN-enabled baseline formulation [1], we consider a multislot industrial image recognition scenario with mobile industrial devices and edge servers. The proposed work adopts the core task, mobility, communication, computation, and delay-energy optimization settings of the baseline environment. To avoid unnecessary repetition, this section summarizes only the essential models required to define the offloading problem, while the proposed GAT-based representation and coverage-aware action masking mechanism are introduced in the solution section.

A. System Overview

We consider a DITEN-enabled IIoT system with a set of mobile industrial devices denoted by

$$\mathcal{D} = \{d_1, d_2, \dots, d_D\}, \quad (1)$$

and a set of edge servers denoted by

$$\mathcal{S} = \{s_1, s_2, \dots, s_S\}. \quad (2)$$

The system operates over a finite set of time slots

$$\mathcal{T} = \{1, 2, \dots, T\}. \quad (3)$$

At each time slot t , each industrial device d_i may generate an industrial image recognition task q_i^t . The edge servers are deployed at fixed locations and provide computation resources for nearby industrial devices. The location of edge server s_j is denoted by

$$L_j^s = [x_j^s, y_j^s]^T, \quad (4)$$

while the location of device d_i at time slot t is denoted by

$$L_i^t = [x_i^t, y_i^t]^T. \quad (5)$$

B. Task Dependency Model

Industrial image recognition is usually composed of multi-ple dependent subtasks, such as image extraction, denoising, standardization, feature extraction, and recognition. Therefore, the task generated by device d_i at time slot t is modeled as a directed acyclic graph (DAG), defined as

$$q_i^t = (\mathcal{V}_i^t, \mathcal{E}_i^t), \quad (6)$$

where

$$\mathcal{V}_i^t = \{v_{i,1}^t, v_{i,2}^t, \dots, v_{i,M}^t\} \quad (7)$$

is the set of M subtasks $\mathcal{V}_i^t$ and $\mathcal{E}_i^t$ is the set of dependency edges among subtasks. If there is an edge $(v_{i,k}^t, v_{i,m}^t) \in \mathcal{E}_i^t$, subtask $v_{i,m}^t$ can only start after its predecessor $v_{i,k}^t$ has been completed and the required intermediate result has been delivered.

Each subtask $v_{i,m}^t$ is characterized by

$$\chi_{i,m}^t = \{C_{i,m}^t, D_{i,m}^t, R_{i,m}^t\}, \quad (8)$$

where $C_{i,m}^t$ denotes the required CPU cycles, $D_{i,m}^t$ denotes the input data size, and $R_{i,m}^t$ denotes the output result size. The predecessor set of subtask $v_{i,m}^t$ is denoted by

$$\text{pred}(v_{i,m}^t) = \{v_{i,k}^t \mid (v_{i,k}^t, v_{i,m}^t) \in \mathcal{E}_i^t\}. \quad (9)$$

For each subtask $v_{i,m}^t$, the offloading decision is denoted by

$$o_{i,m}^t \in \{0, 1, \dots, S\}, \quad (10)$$

where $o_{i,m}^t = 0$ means that subtask $v_{i,m}^t$ is executed locally on device d_i , and $o_{i,m}^t = 1$ means that subtask $v_{i,m}^t$ is offloaded to edge server s_j . For convenience, an entry subtask $v_{i,0}^t$ and an exit subtask $v_{i,M+1}^t$ are introduced in the DAG. These two virtual subtasks have no actual data volume and are executed locally. They are used to represent the start and completion of the whole task.

C. Mobility Model

Since industrial devices may move along predefined routes, their connectivity with edge servers changes over time. Following [1], the movement of each device is modeled in a slotted manner. In time slot t , the position of device d_i is denoted by L_i^t . Each edge server s_j has a fixed signal coverage range.

If device d_i connects to edge server s_j during time slot t , the start moment and end moment of the connection are denoted by $l_{i,j}^{t,start}$ and $l_{i,j}^{t,end}$, respectively. Therefore, if subtask $v_{i,m}^t$ is offloaded to edge server s_j , its edge execution must start after the connection is established and must finish before the connection is lost. These connection-window constraints are expressed as

$$ST_{i,m}^{t,edge}(j) > l_{i,j}^{t,start}, \quad \text{if } o_{i,m}^t = j, \quad (11)$$

and

$$FIN_{i,m}^{t,edge}(j) < l_{i,j}^{t,end}, \quad \text{if } o_{i,m}^t = j. \quad (12)$$

These constraints ensure that an offloaded subtask can be completed within the available communication window between the device and the selected edge server.

D. DT model

In the considered DITEN architecture, the digital twin layer maintains virtual representations of industrial devices, tasks, and edge servers. The digital twin of device d_i at time slot t is represented as

$$DT_{d_i}^t = \{q_i^t, L_i^t, \hat{f}_i^{loc}, W_i^t\}, \quad (13)$$

where q_i^t is the generated task, L_i^t is the device location, $\hat{f}_i^{loc}$ is the estimated local computing capability, and W_i^t is the local waiting delay. Similarly, the digital twin of edge server s_j is defined as

$$DT_{s_j}^t = \{L_j^s, \hat{f}_j^{edge}, W_j^t\}, \quad (14)$$

where $\hat{f}_j^{edge}$ is the estimated computing capability of s_j , and W_j^t is the waiting delay of the edge server. Compared with the full DT model in the baseline [1], the virtual-physical state deviation terms are not explicitly expanded in this report, because the main objective is to evaluate the effect of graph-attention-based state representation and feasibility-aware action selection. Nevertheless, the essential DT states, including task information, device location, computing capability, and waiting delay, are retained in the simulation environment.

E. Communication and Computation Model

When subtask $v_{i,m}^t$ is offloaded to edge server s_j , the input data and intermediate results may need to be transmitted between device d_i and edge server s_j . The uplink transmission rate from device d_i to edge server s_j at time slot t is defined as

$$r_{i,j}^{t,up} = B \log_2 \left(1 + \frac{p_i \cdot g_{i,j}^t}{N_0} \right), \quad (15)$$

where B is the channel bandwidth, p_i is the transmission power of device d_i , $g_{i,j}^t$ is the channel gain between d_i and s_j , and N_0 is the noise power. Similarly, the downlink transmission rate from edge server s_j to device d_i is given by

$$r_{i,j}^{t,down} = B \log_2 \left(1 + \frac{P_j \cdot g_{j,i}^t}{N_0} \right), \quad (16)$$

where P_j is the transmission power of edge server s_j . Here, $g_{i,j}^t$ denotes the channel gain of the wireless link between d_i and s_j . If uplink and downlink channels are modeled separately, $g_{i,j}^t$ can be replaced by $g_{i,j}^{t,up}$ and $g_{j,i}^{t,down}$, respectively.

If subtask $v_{i,m}^t$ is executed locally on device d_i , its local execution delay is calculated as

$$T_{i,m}^{t,loc} = \frac{C_{i,m}^t}{\hat{f}_i^{loc}} + W_i^t, \quad (17)$$

and the corresponding local energy consumption is

$$E_{i,m}^{t,loc} = \kappa_i C_{i,m}^t (\hat{f}_i^{loc})^2, \quad (18)$$

where κ_i is the effective switched capacitance coefficient of device d_i .

If subtask $v_{i,m}^t$ is offloaded to edge server s_j , the uplink transmission delay is

$$T_{i,m,j}^{t,up} = \frac{D_{i,m}^t}{r_{i,j}^{t,up}}, \quad (19)$$

and the edge execution delay is

$$T_{i,m,j}^{t,exe} = \frac{C_{i,m}^t}{\hat{f}_{j,edge}^t} + W_j^t. \quad (20)$$

If the result needs to be returned to device d_i , the downlink transmission delay is

$$T_{i,m,j}^{t,down} = \frac{R_{i,m}^t}{r_{i,j}^{t,down}}. \quad (21)$$

Therefore, the total edge execution delay can be expressed as

$$T_{i,m}^{t,edge}(j) = T_{i,m,j}^{t,up} + T_{i,m,j}^{t,exe} + T_{i,m,j}^{t,down}. \quad (22)$$

The corresponding energy consumption is given by

$$E_{i,m}^{t,edge}(j) = p_i T_{i,m,j}^{t,up} + \kappa_j C_{i,m}^t (\hat{f}_{j,edge}^t)^2 + P_j T_{i,m,j}^{t,down}, \quad (23)$$

where κ_j is the energy coefficient of edge server s_j .

Considering the dependency among subtasks, the start time of each subtask must be later than the completion time of all its predecessor subtasks. Let $ST_{i,m}^t$ and $FIN_{i,m}^t$ denote the start time and finish time of subtask $v_{i,m}^t$, respectively. Then,

$$ST_{i,m}^t \geq FIN_{i,k}^t + \Delta_{k,m}^t, \quad \forall v_{i,k}^t \in \text{pred}(v_{i,m}^t), \quad (24)$$

where $\Delta_{k,m}^t$ denotes the transmission delay of the intermediate result between subtasks $v_{i,k}^t$ and $v_{i,m}^t$, depending on their execution locations. The finish time is computed as

$$FIN_{i,m}^t = \begin{cases} ST_{i,m}^t + T_{i,m}^{t,loc}, & o_{i,m}^t = 0, \\ ST_{i,m}^t + T_{i,m}^{t,edge}(j), & o_{i,m}^t = j. \end{cases} \quad (25)$$

Here, $\Delta_{k,m}^t$ abstracts the intermediate transmission delay between two dependent subtasks and depends on their execution locations. This compact representation is used to keep the formulation concise, while the simulation environment follows the baseline execution logic, including local execution, edge execution, waiting delay, and intermediate data transmission among dependent subtasks.

The total task processing delay of q_i^t is defined as

$$T_i^t = FIN_{i,M+1}^t - FIN_{i,0}^t, \quad (26)$$

and the total energy consumption is defined as

$$E_i^t = \sum_{m=1}^M E_{i,m}^t, \quad (27)$$

where

$$E_{i,m}^t = \begin{cases} E_{i,m}^{t,loc}, & o_{i,m}^t = 0, \\ E_{i,m}^{t,edge}(j), & o_{i,m}^t = j. \end{cases} \quad (28)$$

F. Problem Formulation

Based on the above models, the objective is to determine the execution order and offloading decisions of all subtasks generated by all industrial devices over multiple time slots. The optimization goal is to minimize the weighted sum of task processing delay and energy consumption. Therefore, the dependent-task offloading problem is formulated as

$$\begin{aligned} \mathbf{P1} : \quad & \min_{\{R_{d_i}^t, O_i^t\}} \sum_{t=1}^T \sum_{i=1}^D (\lambda_d T_i^t + \lambda_e E_i^t) \\ \text{s.t.} \quad & ST_{i,m}^{t,edge}(j) > l_{i,j}^{t,start}, \quad \text{if } o_{i,m}^t = j, \\ & FIN_{i,m}^{t,edge}(j) < l_{i,j}^{t,end}, \quad \text{if } o_{i,m}^t = j, \\ & T_i^t \leq T_{\max}, \quad \forall i, t, \\ & o_{i,m}^t \in \{0, 1, \dots, S\}, \quad \forall i, m, t, \\ & E_i^t \leq E_{\max}, \quad \forall i, t. \end{aligned} \quad (29)$$

Here, $R_{d_i}^t$ denotes the execution order of subtasks of device d_i in time slot t , B_i^t denotes the binary offloading decisions, and O_i^t denotes the selected execution locations. λ_d and λ_e are the weighting factors for task processing delay and energy consumption, respectively. $T_{\max}$ is the maximum tolerable delay, and $E_{\max}$ is the maximum acceptable energy consumption.

IV. PROPOSED GAT-ENHANCED OFFLOADING FRAMEWORK

In this section, we present the proposed GAT-Mask MAPPO framework for dependent-task offloading in DITENS. Based on the baseline system model and optimization objective, the proposed framework focuses on improving the state representation and reducing infeasible action exploration. It consists of three main components: a task-level GAT for dependency-aware subtask representation, a topology-level GAT for dynamic device-server representation, and a coverage-aware action mask for feasible action selection.

A. Overview of the Proposed Framework

At each time slot, the industrial image recognition task is represented as a DAG, and the network state is represented as a device-server topology graph. The task-level GAT extracts attention-aware subtask features from the DAG, while the topology-level GAT encodes the current connectivity and resource states between industrial devices and edge servers. These graph-based features are combined with scalar digital-twin states and used as the observation of the MAPPO agent. Before action selection, unavailable edge servers are removed by the coverage-aware action mask, so the policy only samples from feasible local or edge execution actions.

B. Task-Level GAT for Dependency Representation and Priority Generation

For each task $q_i^t = (V_i^t, \mathcal{E}_i^t)$ generated by device d_i , the DAG is used as the task dependency graph. Each subtask $v_{i,m}^t$ is described by its CPU cycles, input data size, and result data size, i.e., $\mathbf{x}_{i,m}^{t,task} = [C_{i,m}^t, D_{i,m}^t, R_{i,m}^t]$. Instead of uniformly aggregating neighboring subtasks, the task-level GAT assigns

different attention weights to different dependency relations and produces an attention-aware embedding:

$$\mathbf{h}_{i,m}^{t,\text{task}} = \text{GAT}_{\text{task}}(\mathbf{x}_{i,m}^{t,\text{task}}, E_i^t). \quad (30)$$

To support sequential offloading decisions, the subtask embedding is mapped into a priority score:

$$p_{i,m}^t = \mathbf{w}_p^T \mathbf{h}_{i,m}^{t,\text{task}} + b_p. \quad (31)$$

The execution priority order is then obtained by sorting subtasks according to their priority scores while preserving DAG precedence constraints:

$$R_{d_i}^t = \text{DAGSort}(V_i^t, E_i^t, \{p_{i,m}^t\}_{m=1}^M). \quad (32)$$

In this way, the task-level GAT keeps the dependency-aware scheduling role of the GCN-based baseline, but replaces uniform neighborhood aggregation with attention-based representation. For a strictly sequential image recognition pipeline, the DAG order still determines feasible execution, while the learned priority scores provide additional importance-aware features for the reinforcement learning policy.

C. Topology-Level GAT for Network Representation

In addition to task dependencies, the offloading decision depends on the dynamic connectivity between industrial devices and edge servers. The topology-level GAT is introduced to encode this device-server relationship without changing the original environment physics. The topology graph is built directly from the joint state of the DITEN environment, including device states, server states, waiting delays, and connection-window information.

At time slot t , the topology graph is defined as

$$\mathcal{G}_{\text{topo}}^t = (\mathcal{D} \cup \mathcal{S}, \mathcal{E}_{\text{topo}}^t), \quad (33)$$

where device nodes contain local computing power, local waiting delay, current subtask information, and accumulated local context. Server nodes contain edge computing power and server waiting delay. A device-to-server edge is created when the connection window between device d_i and server s_j is valid.

For each valid device-server edge, the edge feature is represented as

$$\mathbf{e}_{i,j}^{t,\text{topo}} = [\bar{l}_{i,j}^{t,\text{start}}, \bar{l}_{i,j}^{t,\text{end}}, \bar{\Delta}l_{i,j}^t, \bar{r}_{i,j}^t], \quad (34)$$

where $\bar{l}_{i,j}^{t,\text{start}}$ and $\bar{l}_{i,j}^{t,\text{end}}$ denote the normalized start and end moments of the connection window, $\bar{\Delta}l_{i,j}^t$ denotes the normalized window length, and $\bar{r}_{i,j}^t$ denotes an optional normalized transfer-rate estimate.

The topology-level GAT then produces a topology-aware representation for each device:

$$\mathbf{h}_i^{t,\text{topo}} = \text{GAT}_{\text{topo}}(\mathbf{x}_{d_i}^{t,\text{topo}}, \{\mathbf{x}_{s_j}^{t,\text{topo}}\}_{j=1}^S, \{\mathbf{e}_{i,j}^{t,\text{topo}}\}_{j=1}^S). \quad (35)$$

The resulting representation $\mathbf{h}_i^{t,\text{topo}}$ summarizes the current edge-server availability, resource status, waiting delay, and connection-window information from the perspective of device

d_i . Since the numbers of device nodes and server nodes are fixed by D and S , the graph shape remains stable across time slots. Only the edge availability and edge features change according to the current connection state. Therefore, the topology-level GAT improves the expressiveness of the state representation while preserving the original communication, computation, and mobility models.

D. MDP Formulation

Based on the task-level and topology-level graph representations, the dependent-task offloading problem is formulated as a multi-agent Markov Decision Process (MDP). Each industrial device d_i is regarded as an agent. At each time slot t , the agent sequentially makes offloading decisions for the subtasks of q_i^t according to the priority order $R_{d_i}^t$. The MDP consists of the observation space, action space, reward function, transition probability, and discount factor. The transition follows the original DITEN environment dynamics, while the proposed method changes only the observation representation and feasible action selection.

When device d_i makes an offloading decision for subtask $v_{i,m}^t$, its observation is constructed as

$$\mathbf{o}_{i,m}^t = [\mathbf{h}_{i,m}^{t,\text{task}} \parallel \mathbf{h}_i^{t,\text{topo}} \parallel p_{i,m}^t \parallel \phi_{i,m}^t], \quad (36)$$

where $\mathbf{h}_{i,m}^{t,\text{task}}$ is the task-level GAT embedding, $\mathbf{h}_i^{t,\text{topo}}$ is the topology-level GAT embedding, and $p_{i,m}^t$ is the learned priority score. The vector $\phi_{i,m}^t$ includes the remaining scalar states from the DITEN environment, such as local computing capability, local waiting delay, edge-server computing capabilities, edge-server waiting delays, connection-window information, accumulated delay, accumulated energy consumption, and previous execution context. For centralized training, the joint observation of all agents is formed by collecting $\mathbf{o}_{i,m}^t$ from all devices.

The action of agent d_i is the execution-location decision for the current subtask. The original action space is defined as

$$\mathcal{A}_{i,m}^t = \{0, 1, \dots, S\}, \quad (37)$$

where action 0 represents local execution and action j represents offloading to edge server s_j . Since not all edge servers are available at every time slot, this action space is further restricted by the coverage-aware action mask in the next subsection.

The reward follows the delay-energy optimization objective of the baseline formulation. For subtask $v_{i,m}^t$, the immediate reward is defined as

$$\begin{aligned} r_{i,m}^t = & \lambda_1 \left(\frac{T_{\max}}{M} - T_{i,m}^t \right) + \lambda_2 (T_{\max} - T_{i,\text{acc}}^t) \\ & + \lambda_3 \left(\frac{E_{\max}}{M} - E_{i,m}^t \right) + \lambda_4 (E_{\max} - E_{i,\text{acc}}^t) + \lambda_5 p_{\text{out}} \end{aligned} \quad (38)$$

where $T_{i,m}^t$ and $E_{i,m}^t$ are the delay and energy consumption of the current subtask, respectively. $T_{i,\text{acc}}^t$ and $E_{i,\text{acc}}^t$ denote the accumulated delay and accumulated energy consumption

of the executed subtasks. p_{out} is the penalty for violating the available connection window, and $\lambda_1 - \lambda_5$ are reward weights. During multi-agent training, the global reward can be obtained by aggregating the immediate rewards of all device agents.

This MDP formulation preserves the original delay-energy objective and environment transition logic. The main difference is that the proposed framework uses GAT-enhanced observations and applies action masking to avoid infeasible edge-server selections before policy sampling.

E. Coverage-Aware Action Masking

In dynamic industrial edge environments, not all edge servers are feasible offloading destinations at every time slot. Due to device mobility and limited signal coverage, an industrial device may only connect to a subset of edge servers during the current connection window. If the agent is allowed to select unavailable edge servers, the training process may suffer from invalid exploration and unstable policy updates.

To address this issue, a coverage-aware action masking mechanism is introduced. For device d_i and edge server s_j at time slot t , the connection availability indicator is defined as

$$c_{i,j}^t = \begin{cases} 1, & \text{if } s_j \text{ is available for } d_i \text{ at time slot } t, \\ 0, & \text{otherwise.} \end{cases} \quad (39)$$

Based on the connection availability, the valid action set for subtask $v_{i,m}^t$ is defined as

$$\mathcal{A}_{i,m}^{t,\text{valid}} = \{0\} \cup \{j \mid c_{i,j}^t = 1, j \in \{1, 2, \dots, S\}\}. \quad (40)$$

The local execution action is always retained because the device can always execute the subtask locally. In contrast, an edge-server action is retained only when the corresponding server is currently available for the device.

For a policy-based reinforcement learning agent, let $\pi_\theta(\mathbf{o}_{i,m}^t)$ denote the action probability distribution generated from the observation $\mathbf{o}_{i,m}^t$. Before action selection, infeasible actions are masked by assigning zero probability to unavailable edge servers:

$$\tilde{\pi}_\theta(a \mid \mathbf{o}_{i,m}^t) = \begin{cases} \pi_\theta(a \mid \mathbf{o}_{i,m}^t), & a \in \mathcal{A}_{i,m}^{t,\text{valid}}, \\ 0, & a \notin \mathcal{A}_{i,m}^{t,\text{valid}}. \end{cases} \quad (41)$$

The masked policy is then normalized as

$$\bar{\pi}_\theta(a \mid \mathbf{o}_{i,m}^t) = \frac{\tilde{\pi}_\theta(a \mid \mathbf{o}_{i,m}^t)}{\sum_{a' \in \mathcal{A}_{i,m}^{t,\text{valid}}} \tilde{\pi}_\theta(a' \mid \mathbf{o}_{i,m}^t)}. \quad (42)$$

The final action is selected from the masked policy:

$$a_{i,m}^t \sim \bar{\pi}_\theta(a \mid \mathbf{o}_{i,m}^t). \quad (43)$$

In implementation, the same operation can be performed at the logit level by assigning a very small value, such as $-\infty$, to infeasible actions before applying the softmax function. This prevents unavailable edge servers from being sampled while preserving differentiability for valid actions.

The action mask does not change the original delay-energy optimization objective or the environment dynamics. Instead,

it restricts the policy to physically feasible actions based on the current coverage state. Therefore, the agent can focus on valid local and edge execution choices, which improves learning efficiency and reduces invalid offloading decisions.

F. Masked GAT-MAPPO Training Procedure

Based on the GAT-enhanced observation and coverage-aware action mask, the proposed offloading policy is trained using a masked multi-agent proximal policy optimization framework. Unlike off-policy methods that rely on a replay buffer and target networks, MAPPO collects on-policy trajectories from the current policy and updates the actor and critic networks using the clipped PPO objective.

At each decision step, the task-level GAT extracts the subtask representation and priority score, while the topology-level GAT extracts the device-server topology representation. The observation $\mathbf{o}_{i,m}^t$ is then constructed for each device agent. Before action sampling, the coverage-aware mask is applied to the policy logits so that unavailable edge servers cannot be selected. The log probability of the selected action is computed after masking and stored together with the observation, action, reward, value estimate, and action mask.

For centralized training, the value function can use the joint observation $\mathbf{O}_m^t$ of all device agents:

$$V_\psi(\mathbf{O}_m^t), \quad (44)$$

where ψ denotes the parameters of the value network. The policy network is parameterized by θ and generates the masked policy distribution

$$\bar{\pi}_\theta(a_{i,m}^t \mid \mathbf{o}_{i,m}^t). \quad (45)$$

Let n denote a collected decision step in the rollout buffer. The probability ratio used in PPO is defined as

$$\rho_n(\theta) = \frac{\bar{\pi}_\theta(a_n \mid \mathbf{o}_n)}{\bar{\pi}_{\theta_{\text{old}}}(a_n \mid \mathbf{o}_n)}. \quad (46)$$

The clipped policy objective is defined as

$$L^{\text{clip}}(\theta) = \mathbb{E}_n \left[\min \left(\rho_n(\theta) \hat{A}_n, \text{clip}(\rho_n(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_n \right) \right], \quad (47)$$

where $\hat{A}_n$ is the estimated advantage and ϵ is the PPO clipping parameter.

The value loss is computed as

$$L^{\text{value}}(\psi) = \mathbb{E}_n \left[\left(V_\psi(\mathbf{O}_n) - \hat{G}_n \right)^2 \right], \quad (48)$$

where $\hat{G}_n$ is the discounted return. The total optimization objective is

$$L(\theta, \psi) = -L^{\text{clip}}(\theta) + c_v L^{\text{value}}(\psi) - c_e \mathcal{H}(\bar{\pi}_\theta), \quad (49)$$

where c_v and c_e are the coefficients of the value loss and entropy bonus, respectively, and $\mathcal{H}(\bar{\pi}_\theta)$ denotes the entropy of the masked policy.

The overall training process is summarized in Algorithm IV-F.


```

1: Initialize task-level GAT, topology-level GAT, policy network  $\pi_\theta$ , value network  $V_\psi$ , and rollout buffer  $\mathcal{B}$ .
2: for episode  $= 1, 2, \dots, E$  do
3:   Initialize the DITEN environment.
4:   for time slot  $t = 1, 2, \dots, T$  do
5:     Generate task DAGs  $\{q_i^t\}_{i=1}^D$  for all devices.
6:     Construct the topology graph  $\mathcal{G}_{\text{topo}}^t$  from the current joint state.
7:     for each device  $d_i$  do
8:       Compute task-level GAT embeddings  $\mathbf{h}_{i,m}^{t,\text{task}}$  and priority scores  $p_{i,m}^t$ .
9:       Generate the DAG-constrained priority order  $R_{d_i}^t$ .
10:      Compute topology-level GAT embedding  $\mathbf{h}_i^{t,\text{topo}}$ .
11:    end for
12:    for each subtask decision step  $m = 1, 2, \dots, M$  do
13:      for each device agent  $d_i$  do
14:        Construct observation  $\mathbf{o}_{i,m}^t$ .
15:        Build valid action set  $\mathcal{A}_{i,m}^{t,\text{valid}}$  using the coverage-aware mask.
16:        Generate masked policy  $\bar{\pi}_\theta(a | \mathbf{o}_{i,m}^t)$ .
17:        Sample action  $a_{i,m}^t$  from the masked policy.
18:      end for
19:      Execute joint actions and observe rewards, next observations, and done signals.
20:      Store observations, actions, masks, rewards, values, and log probabilities in  $\mathcal{B}$ .
21:    end for
22:  end for
23:  Compute returns  $\hat{G}_n$  and advantages  $\hat{A}_n$  from the collected rollout.
24:  for optimization epoch  $= 1, 2, \dots, K$  do
25:    Update policy network  $\pi_\theta$  using the clipped PPO objective.
26:    Update value network  $V_\psi$  by minimizing the value loss.
27:  end for
28:  Clear rollout buffer  $\mathcal{B}$ .
29: end for

```

V. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental Settings

The experiments are conducted in a DITEN-based industrial image recognition simulation environment following the main settings of the GCN-enabled baseline. The scenario contains 10 mobile industrial devices and 3 edge servers. Each episode consists of 50 time slots, and each device generates a dependent image recognition task with 5 subtasks at each time slot. The devices move at a speed of 1 m/s, and the edge-server coverage radius is set to 12 m. The communication bandwidth is 10 MHz. The transmission powers of industrial devices and edge servers are set to 0.5 W and 1.2 W, respectively. The main experimental parameters are summarized in Table I.

TABLE I
MAIN EXPERIMENTAL SETTINGS

Parameter	Value
Number of industrial devices	10
Number of edge servers	3
Number of time slots	50
Slot duration	1 s
Number of subtasks per task	5
Device speed	1 m/s
Coverage radius	12 m
Bandwidth	10 MHz
Device transmission power	0.5 W
Server transmission power	1.2 W
Training episodes	500
RL learning rate	0.0001

TABLE II
COMPARED METHODS

Method	Description
MADDPG	Standard MADDPG baseline.
MAAC	Multi-agent actor-critic baseline.
MAPPO	Standard MAPPO without GAT or action mask.
Mask-MAPPO	MAPPO with coverage-aware action masking.
GAT-Mask MAPPO	Proposed full model with GAT and action masking.

B. Compared Methods

To evaluate the proposed method, we compare it with several multi-agent reinforcement learning baselines and ablation variants, as summarized in Table II. Local Only and Edge Only are used as simple reference policies to show the limitations of pure local execution and pure edge offloading. MADDPG, MAAC, and MAPPO are adopted as learning-based baselines. Since the proposed method is built upon MAPPO, the standard MAPPO model is used as the main comparison baseline. The proposed full model is denoted as GAT-Mask MAPPO.

C. Ablation Study Setup

To further analyze the contribution of the proposed components, we conduct an ablation study using three MAPPO-based variants. MAPPO is used as the basic policy optimization baseline without GAT representation or action masking. Mask-MAPPO evaluates the effect of coverage-aware action masking by removing unavailable edge servers before action selection. GAT-Mask MAPPO combines GAT-enhanced representation and coverage-aware action masking, and represents the full proposed model. This comparison helps evaluate the effect of action masking and the additional benefit of GAT-based state representation.

D. Overall Performance Comparison

The overall performance comparison is shown in Table III. The reported results are taken from the final available evaluation logs. For fixed policies, the results are reported at episode 5, while learning-based methods are reported at episode 500.

As shown in Table III, the proposed GAT-Mask MAPPO achieves the best overall performance among all compared



Fig. 1. Performance comparison of different methods in terms of reward, task processing delay, and energy consumption.

TABLE III
OVERALL PERFORMANCE COMPARISON

Method	Reward	Delay (s)	Energy (J)
Local Only	3.287	1.392	0.101
Edge Only	1.539	1.454	0.130
MADDPG	2.932	1.268	0.120
MAAC	3.750	1.208	0.101
MAPPO	3.730	1.216	0.101
Mask-MAPPO	4.099	0.943	0.078
GAT-Mask MAPPO	4.105	0.936	0.076

methods. Compared with the standard MAPPO, GAT-Mask MAPPO improves the reward from 3.730 to 4.105, reduces the task processing delay from 1.216 s to 0.936 s, and decreases the energy consumption from 0.101 J to 0.076 J. This corresponds to approximately 10.05% higher reward, 22.97% lower delay, and 24.58% lower energy consumption. These results indicate that combining graph-attention-based representation with coverage-aware action masking can help the agent make more efficient offloading decisions in dynamic DITEN environments.

The results also show that simply using local execution or aggressive edge offloading is not sufficient in the considered dynamic DITEN scenario. Local Only avoids infeasible offloading actions, but it cannot fully exploit the computing resources of edge servers. In contrast, Edge Only suffers from limited coverage and connection-window constraints, which leads to many infeasible edge requests and a lower reward. Standard learning-based methods such as MAAC and MAPPO achieve more stable performance, but they still tend to select local execution in many cases to avoid invalid offloading penalties. By introducing coverage-aware action masking, Mask-MAPPO can safely explore feasible edge execution choices. Furthermore, GAT-Mask MAPPO achieves the best performance because it combines feasible action selection with graph-enhanced state representation, allowing the agent to better utilize edge resources while maintaining low delay and energy consumption.

E. Ablation Study

The ablation results can be observed from the comparison among MAPPO, Mask-MAPPO, and GAT-Mask MAPPO. Compared with MAPPO, Mask-MAPPO significantly improves the reward and reduces both delay and energy con-

sumption, showing that coverage-aware action masking is effective in avoiding infeasible offloading decisions. Furthermore, GAT-Mask MAPPO achieves slightly better performance than Mask-MAPPO, reducing the delay from 0.943 s to 0.936 s and the energy consumption from 0.078 J to 0.076 J. This confirms that the GAT-based task and topology representations provide additional benefits by enhancing the quality of the state representation. Overall, the full model achieves the best trade-off among reward, delay, and energy consumption.

VI. CONCLUSION AND LIMITATIONS

In this report, we investigated the dependent-task offloading problem for industrial image recognition in Digital Twin Edge Networks (DITENs). Following the core simulation environment and delay-energy optimization setting of the GCN-enabled baseline, we proposed a GAT-Mask MAPPO framework to improve offloading decision-making in dynamic industrial edge environments. The proposed method integrates task-level graph attention, topology-level graph attention, and coverage-aware action masking. The task-level GAT is used to enhance the representation of subtask dependencies, while the topology-level GAT captures device-server connectivity and resource-related information from the current joint state. In addition, the action mask prevents the policy from selecting unavailable edge servers, thereby reducing invalid exploration during training.

Experimental results show that the proposed GAT-Mask MAPPO achieves the best overall performance among the evaluated methods. Compared with the standard MAPPO baseline, GAT-Mask MAPPO improves the reward from 3.730 to 4.105, reduces task processing delay from 1.216 s to 0.936 s, and decreases energy consumption from 0.101 J to 0.076 J. The comparison between MAPPO and Mask-MAPPO indicates that coverage-aware action masking plays an important role in improving the feasibility and efficiency of offloading decisions. Moreover, the improvement of GAT-Mask MAPPO over Mask-MAPPO further suggests that graph-attention-based representations can provide additional benefits by enhancing the state information used by the policy. Overall, the results demonstrate that combining graph attention with feasibility-aware action selection is effective for delay-energy-aware task offloading in DITEN-based industrial scenarios.

However, this work still has several limitations. First, the experiments are conducted in a simulation environment based on the baseline setting, and the proposed method has not yet been validated in a real industrial edge network. Second, although the topology-level GAT improves state representation, it does not change the underlying communication, computation, or mobility models; therefore, its performance still depends on the quality of the simulated environment and available joint-state information. Third, the current ablation study mainly evaluates the effects of action masking and GAT-enhanced representation at the policy level, but it does not separately analyze the individual contributions of task-level GAT and topology-level GAT. In future work, more detailed ablation studies, real-world deployment, task migration, and multi-hop offloading can be further investigated to improve the robustness and applicability of the proposed framework.

REFERENCES

- [1] L. Liao, M. Tao, A. Dong, R. Xie, and Y. Zhang, "Graph-convolutional-network-enabled task offloading for industrial image recognition in digital twin edge networks," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 29 176–29 188, 2025.
- [2] M. Satyanarayanan, "The emergence of edge computing," *computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [3] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE communications surveys & tutorials*, vol. 22, no. 2, pp. 869–904, 2020.
- [4] M. Tao, K. Ota, M. Dong, and H. Yuan, "Stackelberg game-based pricing and offloading in mobile edge computing," *IEEE Wireless Communications Letters*, vol. 11, no. 5, pp. 883–887, 2022.
- [5] L. Tan, Z. Kuang, L. Zhao, and A. Liu, "Energy-efficient joint task offloading and resource allocation in ofdma-based collaborative edge computing," *IEEE Transactions on Wireless Communications*, vol. 21, no. 3, pp. 1960–1972, 2022.
- [6] W. Sun, H. Zhang, R. Wang, and Y. Zhang, "Reducing offloading latency for digital twin edge networks in 6g," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 10, pp. 12 240–12 251, 2020.
- [7] L. X. Nguyen, Y. K. Tun, T. N. Dang, Y. M. Park, Z. Han, and C. S. Hong, "Dependency tasks offloading and communication resource allocation in collaborative uav networks: A metaheuristic approach," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 9062–9076, 2023.
- [8] M. Mehrabi, S. Shen, V. Latzko, Y. Wang, and F. H. Fitzek, "Energy-aware cooperative offloading framework for inter-dependent and delay-sensitive tasks," in *GLOBECOM 2020-2020 IEEE Global Communications Conference*. IEEE, 2020, pp. 1–6.
- [9] C. Xu, M. Lv, K. Zhang, K. Cao, G. Wang, M. Wei, and B. Peng, "Energy consumption and time-delay optimization of dependency-aware tasks offloading for industry 5.0 applications," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 1590–1600, 2023.
- [10] G. Zhang, W. Zhang, Y. Cao, D. Li, and L. Wang, "Energy-delay tradeoff for dynamic offloading in mobile-edge computing system with energy harvesting devices," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 10, pp. 4642–4655, 2018.
- [11] H. Liu, W. Huang, D. I. Kim, S. Sun, Y. Zeng, and S. Feng, "Towards efficient task offloading with dependency guarantees in vehicular edge networks through distributed deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 9, pp. 13 665–13 681, 2024.
- [12] X. Tang, X. Li, R. Yu, Y. Wu, J. Ye, F. Tang, and Q. Chen, "Digital-twin-assisted task assignment in multi-uav systems: A deep reinforcement learning approach," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 15 362–15 375, 2023.
- [13] W. Liu, Y. Fu, Y. Guo, F. Lee Wang, W. Sun, and Y. Zhang, "Two-timescale synchronization and migration for digital twin networks: A multi-agent deep reinforcement learning approach," *IEEE Transactions on Wireless Communications*, vol. 23, no. 11, pp. 17 294–17 309, 2024.
- [14] H. Zhou, L. Chen, K. Jiang, and Y. Wu, "Energy-efficient dependency-aware task offloading in mobile edge computing: A digital twin empowered approach," in *2022 IEEE 10th International Conference on Smart City and Informatization (iSCI)*, 2022, pp. 57–62.

Graph_Attention_and_DRL_for_Task_Offloading_in_Industrial_Image_Recognition_over_Dynamic_
(1).pdf

ORIGINALITY REPORT

5%
SIMILARITY INDEX

1%
INTERNET SOURCES

5%
PUBLICATIONS

0%
STUDENT PAPERS

PRIMARY SOURCES

1 Lingling Liao, Ming Tao, Ani Dong, Renping Xie, Yin Zhang. "Graph-Convolutional-Netwo
Enabled Task Offloading for Industrial Image Recognition in Digital Twin Edge Networks
IEEE Internet of Things Journal, 2025

Publication

2 Wenjing Xiao, Shixin Chen, Xin Ling, Miaojiang Chen, Min Chen, Ahmed Farouk.
"Countering AI-Powered Threats in IoV: A Secure Task Offloading and Resource Allocati
Scheme with RIS and Quantum Reinforcement Learning", IEEE Transactions on Consum
Electronics, 2026

Publication

Exclude quotes
Exclude bibliography

Off
On

Exclude matches < 1%